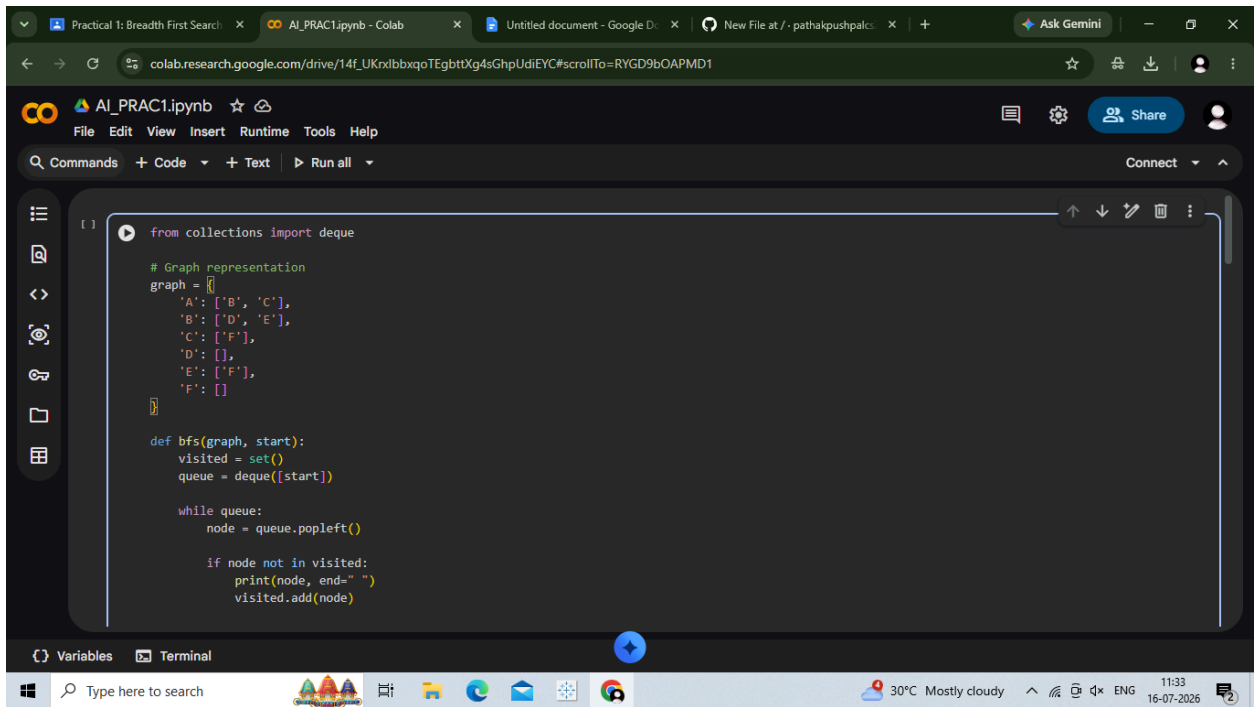


Practical 1: Breadth First Search & Iterative Depth First Search in Python.



The screenshot shows a Google Colab notebook titled "AI_PRAC1.ipynb". The code in the first cell defines a graph and a breadth-first search (bfs) function. The graph is represented as a dictionary where keys are nodes and values are lists of adjacent nodes. The bfs function uses a queue to traverse the graph starting from a specified node.

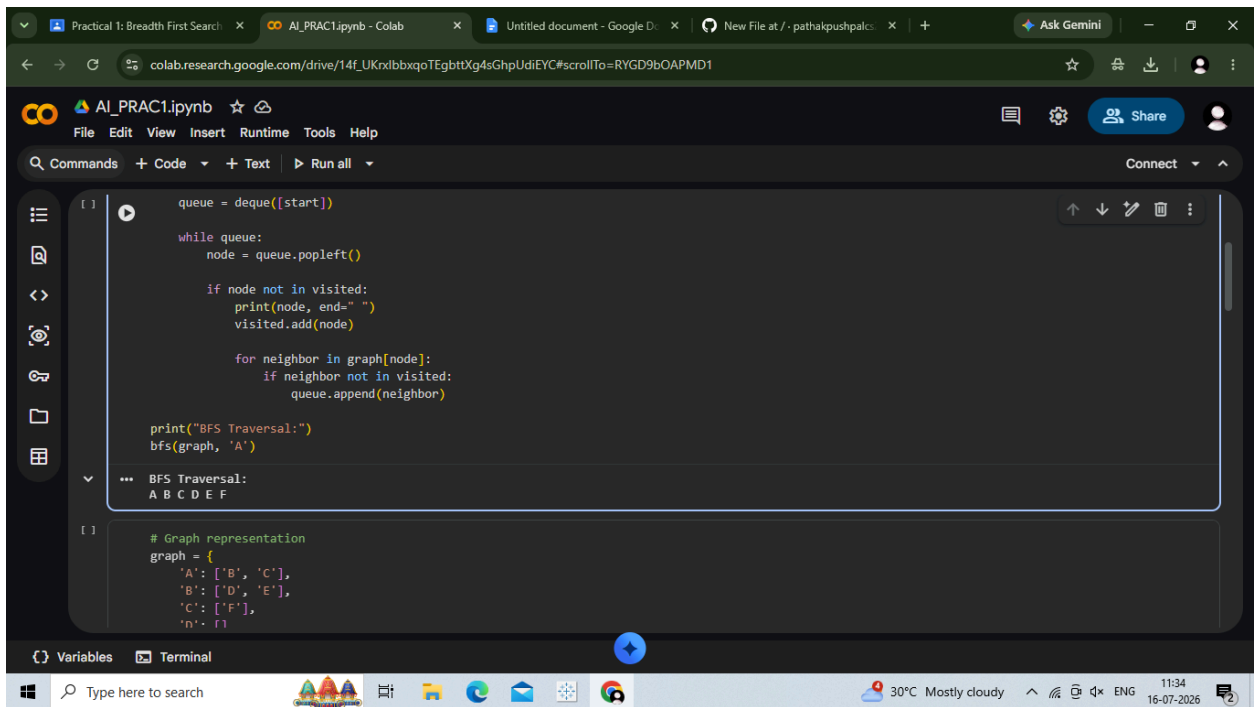
```
[ ] ▶ from collections import deque

# Graph representation
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

def bfs(graph, start):
    visited = set()
    queue = deque([start])

    while queue:
        node = queue.popleft()

        if node not in visited:
            print(node, end=" ")
            visited.add(node)
```



The screenshot shows the same Google Colab notebook, but now the code is executed. The output of the bfs function is displayed, showing the traversal order: A B C D E F. The code in the second cell prints the result of the bfs function call.

```
[ ] ▶ queue = deque([start])

while queue:
    node = queue.popleft()

    if node not in visited:
        print(node, end=" ")
        visited.add(node)

    for neighbor in graph[node]:
        if neighbor not in visited:
            queue.append(neighbor)

print("BFS Traversal:")
bfs(graph, 'A')

... BFS Traversal:
A B C D E F

[ ] ▶ # Graph representation
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}
```

colab.research.google.com/drive/14f_UKx1lbbxqoTEgbtXg4sGhpUdiEYC#scrollTo=RYGD9bOAPMD1

Commands + Code + Text ▶ Run all

Connect

BFS Traversal:
A B C D E F

```
[1] # Graph representation
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

def dfs(graph, start):
    visited = set()
    stack = [start]

    while stack:
        node = stack.pop()

        if node not in visited:
            print(node, end=" ")
            visited.add(node)

            # Reverse to maintain left-to-right traversal
            for neighbor in reversed(graph[node]):
                if neighbor not in visited:
                    stack.append(neighbor)
```

Variables Terminal

Type here to search 30°C Mostly cloudy 11:34 16-07-2026

colab.research.google.com/drive/14f_UKx1lbbxqoTEgbtXg4sGhpUdiEYC#scrollTo=RYGD9bOAPMD1

Commands + Code + Text ▶ Run all

Connect

```
[1] stack = [start]

while stack:
    node = stack.pop()

    if node not in visited:
        print(node, end=" ")
        visited.add(node)

        # Reverse to maintain left-to-right traversal
        for neighbor in reversed(graph[node]):
            if neighbor not in visited:
                stack.append(neighbor)

print("DFS Traversal:")
dfs(graph, 'A')
```

DFS Traversal:
A B D E F C

```
[1] from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
```

Variables Terminal

Type here to search 30°C Mostly cloudy 11:34 16-07-2026

Practical 1: Breadth First Search x AI_PRACT1.ipynb - Colab x Untitled document - Google D... x New File at / - pathakpushpalcs... x + Ask Gemini

colab.research.google.com/drive/14f_UKxllbbxqoTEgbitXg4sGhpUdiEYC#scrollTo=RYGD9bOAPMD1

Commands + Code + Text Run all Connect

DFS Traversal:
A B D E F C

```
[ ] from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

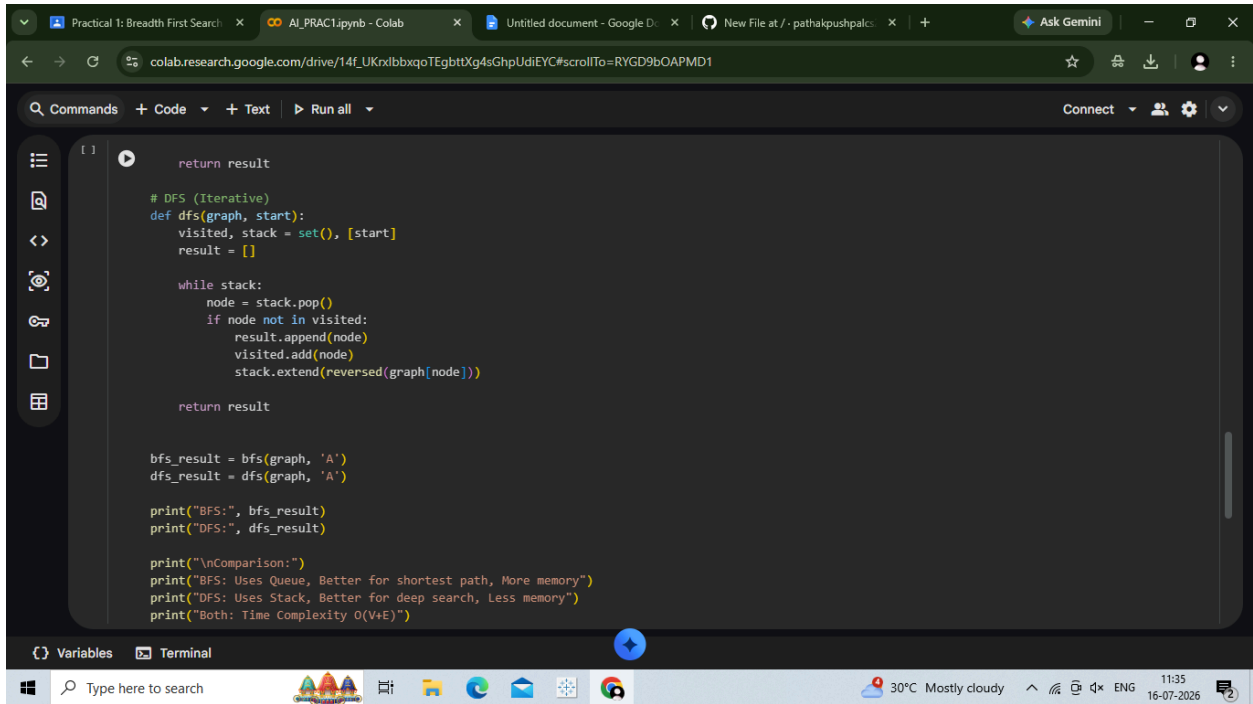
# BFS
def bfs(graph, start):
    visited, queue = set(), deque([start])
    result = []

    while queue:
        node = queue.popleft()
        if node not in visited:
            result.append(node)
            visited.add(node)
            queue.extend(graph[node])

    return result
```

Variables Terminal

Type here to search 30°C Mostly cloudy 11:34 16-07-2025



```
return result

# DFS (Iterative)
def dfs(graph, start):
    visited, stack = set(), [start]
    result = []

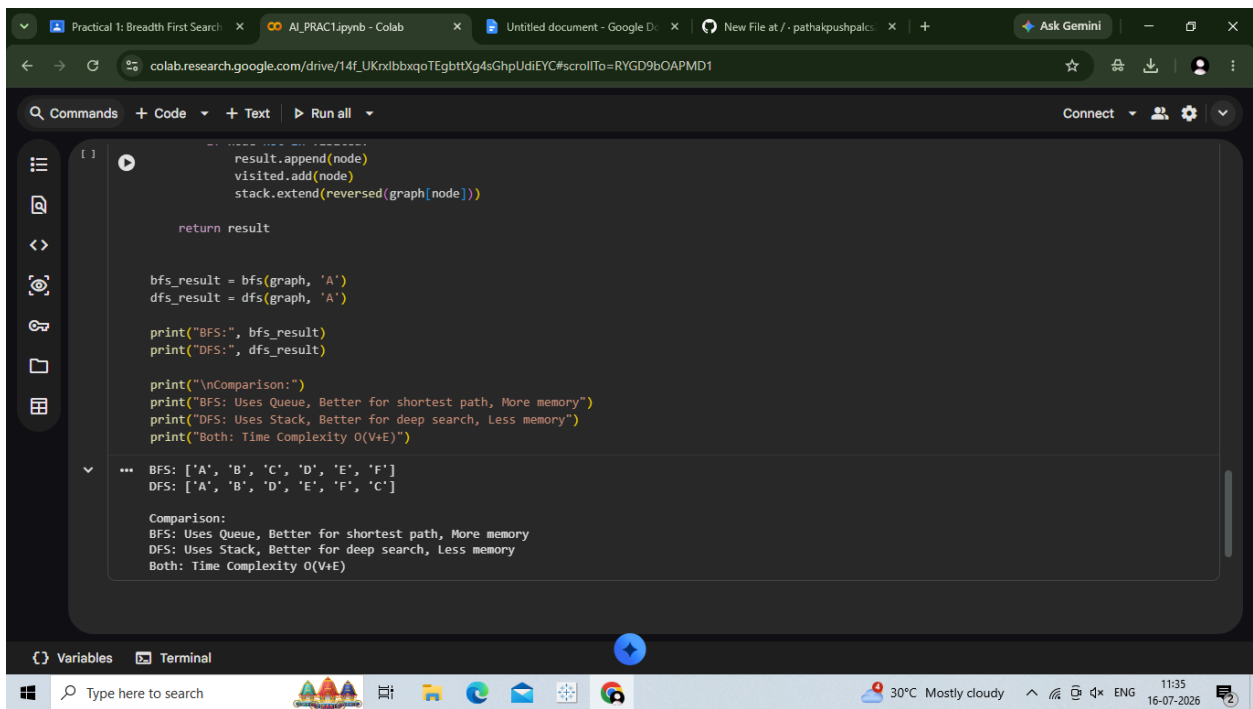
    while stack:
        node = stack.pop()
        if node not in visited:
            result.append(node)
            visited.add(node)
            stack.extend(reversed(graph[node]))

    return result

bfs_result = bfs(graph, 'A')
dfs_result = dfs(graph, 'A')

print("BFS:", bfs_result)
print("DFS:", dfs_result)

print("\nComparison:")
print("BFS: Uses Queue, Better for shortest path, More memory")
print("DFS: Uses Stack, Better for deep search, Less memory")
print("Both: Time Complexity O(V+E)")
```



```
result.append(node)
visited.add(node)
stack.extend(reversed(graph[node]))

return result

bfs_result = bfs(graph, 'A')
dfs_result = dfs(graph, 'A')

print("BFS:", bfs_result)
print("DFS:", dfs_result)

print("\nComparison:")
print("BFS: Uses Queue, Better for shortest path, More memory")
print("DFS: Uses Stack, Better for deep search, Less memory")
print("Both: Time Complexity O(V+E)")

... BFS: ['A', 'B', 'C', 'D', 'E', 'F']
DFS: ['A', 'B', 'D', 'E', 'F', 'C']

Comparison:
BFS: Uses Queue, Better for shortest path, More memory
DFS: Uses Stack, Better for deep search, Less memory
Both: Time Complexity O(V+E)
```